strategynet.ai

Volt: Strategy Search

Finding useful portfolios without evaluating every candidate

RESEARCH ARCHITECTURE • VERSION 0.2

research@strategynet.ai

Open-source working paper • July 2026

ABSTRACT

An option chain can produce more valid portfolios than a researcher can price, simulate, and review. Many candidates are duplicates or slight variations; others look attractive only because their features used stale information or their labels ignored executable prices. Exhaustive evaluation wastes a limited research budget, while an unconstrained ranker can learn the wrong lesson very efficiently.

This paper describes a search process that spends evaluation effort on distinct, legal, point-in-time candidates. Each candidate becomes a versioned Volt graph linked to its exact portfolio identity, bounded actions, and deterministic simulation evidence. The proposed statistical layer learns over those graphs and adds dataset construction, candidate scoring, active acquisition, baseline comparisons, quote-backed labels, and chronological evaluation. The public repository ships synthetic fixtures and semantic reduction results; it does not ship a trained ranker or make an alpha claim. The initial `cog` compiler, compiler-bound graph dataset, and structural legal mask are implemented; account-aware margin and capital policy remain integration work.

Contents

I Research Design **3**
1 Discovery task 3
2 Artifact contract 3
3 Candidate pool 4
II Models **5**
4 Graph view 5
4.1 Schema 5
4.2 Encoding 5
5 Training objective 5
6 Baselines 5
7 Acquisition loop 6
III Evaluation **7**
8 Quote-backed labels 7
9 Splits and metrics 7
10 Research sequence 7
10.1 Canonical search 7
10.2 Event portfolios 7
10.3 Volatility dynamics 7
10.4 Conformance 7
11 Evidence and limits 8
12 Conclusion 8

PART I

Research Design

1 Discovery task

At a decision time t, the research system receives a market and account state together with the contracts that were listed and observable at that time. The implemented `CandidateGrammar` resolves typed actions against the caller-supplied listed universe and enforces structural and exercise-authority rules. Caller policy must still apply liquidity, capital, margin, and account constraints. The implemented `cog` compiler coordinates the structural decisions and their diagnostics; the discovery model should see candidates only after every deterministic decision has been recorded.

The initial task is bounded. Given an admissible set A_t, estimate the outcome and risk of each candidate and rank the set under a declared objective. A candidate may be a complete portfolio constructed from an empty account or a one-step transformation of existing inventory. Either formulation yields an ordinary supervised sample once the simulator or historical replay has supplied the outcome. Exhaustive evaluation remains possible on small universes, which gives the experiment a known oracle and makes search efficiency measurable.

This separation keeps the research question precise. Contract validity, exercise authority, and settlement remain Volt runtime concerns; account admissibility remains an application concern. The model is judged on how effectively it orders legal alternatives, how much expensive evaluation it saves, and whether its ranking remains useful after spreads, fees, and out-of-time market changes.

2 Artifact contract

The proposed research interface exposes a sample

$$d_t = (G_t, A_t, M_t, y_t, p_t),$$

where G_t is the compiled graph, A_t contains feasible candidates, M_t is the legal mask, y_t carries outcome and risk labels, and p_t records provenance. Every element is versioned. The model may replace its graph schema, normalizers, features, and objective without changing the authoritative portfolio or the events that produced its label. The current `PortfolioGraphSample` implements G_t, exact and payoff hashes, and an optional compact target. Volt does not yet serialize the complete five-part sample as one artifact.

Field	Financial content	Experimental use
G_t	portfolio, exact option positions, underlying, expiry, and typed relations	model input and attribution anchor
A_t	resolved candidate portfolios or transformations	objects to score under one acquisition budget
M_t	authority, lifecycle, liquidity, and resource decisions	exclusion before ranking or normalization
y_t	P&L, drawdown, tail risk, turnover, costs, and scenario outcomes	regression, ranking, calibration, or Pareto targets
p_t	timestamps, sources, hashes, assumptions, and split membership	replay, leakage checks, and result identity

The experiment manifest fixes the observation clock, prediction horizon, eligible grammar, execution models, scenario distribution, random seeds, objective, acquisition budget, and chronological split before fitting begins. Normalizers and learned categorical vocabularies use training data alone. Validation data select configurations; the test split remains sealed until the configuration and simulator budget have been frozen.

3 Candidate pool

The public fixtures use six synthetic same-expiry contracts. In compiled mode, the `graph` example constructs and compiles one versioned source program for each quantity vector in $\{-1, 0, 1\}$, producing all $3^6 = 729$ compiler-bound graph samples. Each sample carries source and artifact hashes, a deterministic identifier, an exact portfolio hash, an optional terminal-payoff hash, typed nodes and edges, and a synthetic target.

The `zap` example supplies two semantic controls. Two ordered actions produce 144 raw sequences but 73 exact portfolios. Independently, quantities in $\{-2, -1, 0, 1, 2\}$ across six contracts produce 15,625 exact portfolios and 13,321 terminal-payoff classes. These fixtures test identity, canonicalization, and export plumbing; they contain no historical market cohort, trained model, liquidity filter, or economic claim.

As the universe expands, exhaustive labels become too costly. Candidate generation and evaluation must then be treated as separate budgets. The `cog` compiler or caller application may enumerate or sample a large valid pool cheaply, while the acquisition policy chooses the smaller subset that receives scenario simulation or historical replay.

PART II

Models

4 Graph view

4.1 Schema

The implemented graph is derived from a `CanonicalPortfolio` plus a validated `GraphContext`. Its node types are `Portfolio`, `Underlying`, `Expiry`, and `OptionPosition`; its edge types are `PositionIn`, `WrittenOn`, and `ExpiresAt`. Every sample carries both exact-portfolio and terminal-payoff hashes.

Position features include contract identity, right, settlement, exercise style, strike, declared mark, signed quantity, multiplier, first exercise time, and discrete opportunity count. Context supplies one underlying, currency, valuation time, common expiry, reference spot, and a mark for every contract. Account nodes, Greeks, spread, liquidity, lifecycle state, market events, and multi-expiry graphs are not part of schema version `volt.portfolio_graph.v1`.

The current view accommodates variable leg counts while remaining insensitive to canonical input order. Future experiments may derive richer graph schemas without rewriting contract semantics, but each schema requires its own version and controls. A set encoder and conventional engineered features remain essential comparisons [2].

4.2 Encoding

Type-specific input transforms map nodes into a common latent dimension. Relation-specific message passing produces contextual node states, and a permutation-invariant readout gives a portfolio representation h_t. Candidate a receives its own representation $e(a)$, either from a separately compiled graph or from the proposed graph edit. One useful score is

$$Q_\theta(G_t, a) = f_\theta(h_t, e(a), h_t \odot e(a)) .$$

Companion heads may estimate an outcome distribution, named risk measures, and a behavioral embedding. The legal mask is applied before ranking or probability normalization, so an invalid action cannot acquire probability through a numerical accident.

5 Training objective

No single scalar captures every use of an option portfolio. A research dataset may therefore derive a vector

$$y = (\text{net P\&L}, \text{drawdown}, \text{CVaR}, \text{turnover},\\ \text{capital use}, \text{exercise loss}, \text{scenario vector}),$$

from `SimulationResult` and additional named risk models. The current result directly records fills, journals, pathwise and terminal P&L, leg attribution, final state, and reproducibility hashes; it does not calculate drawdown, CVaR, or capital use automatically. The manifest states how the experiment derives and uses any added targets. A fitting loss may combine outcome regression, pairwise or listwise ranking, distribution calibration, and auxiliary risk targets:

$$\mathcal{L} = \lambda_v \mathcal{L}_{\text{value}} + \lambda_r \mathcal{L}_{\text{rank}} + \lambda_c \mathcal{L}_{\text{calibration}} + \lambda_a \mathcal{L}_{\text{aux}}.$$

Coefficients, target transforms, and any scalar utility belong to the experiment record. Changing them creates a new study while leaving candidate identity and financial labels intact.

6 Baselines

Graph models must justify their additional machinery against simpler alternatives. Every comparison includes deterministic trading heuristics, linear or tree models on engineered features, a permutation-invariant set encoder, and random or unguided acquisition. All models receive identical point-in-time information, candidate pools, labels, and evaluation budgets. Wall-clock measurements accompany simulator-call counts because a model that saves evaluations but consumes much more compute may still be unattractive.

Deep Sets provides the closest structural control: it handles variable-sized, order-invariant collections without relation-specific message passing [1]. GraphGym-style sweeps can standardize optimization and architecture comparisons [4]; PyTorch Geometric supplies the implementation substrate [3]. The Volt manifest continues to own financial objectives, data splits, and evidence identities.

7 Acquisition loop

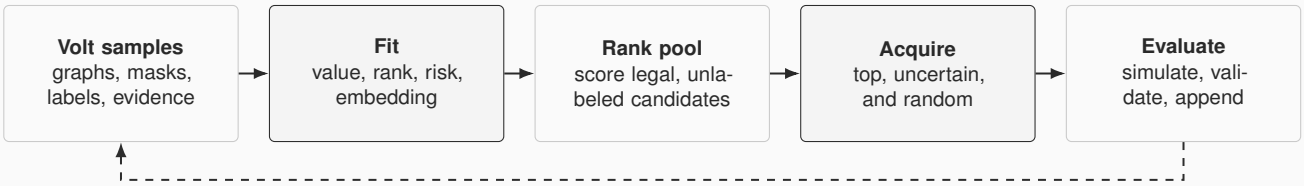


Figure 1. Supervised discovery under a fixed acquisition budget. The feedback path carries immutable simulator evidence into a new dataset version; it never alters an existing label.

An initial labeled cohort fits the value and ranking heads. The model then scores an unlabeled pool, after which the acquisition rule selects candidates for expensive evaluation. High predicted utility alone is insufficient: early errors can concentrate the dataset around an accidental pattern. The batch therefore combines promising candidates with uncertainty or diversity coverage and a compulsory random allocation.

Each acquisition records the candidate universe, model checkpoint, score, selection reason, and probability where available. Once evaluated, the new evidence is appended under a new dataset manifest and the model is retrained. Learning curves plot result quality against both simulator calls and elapsed compute. The experiment stops at its preregistered budget rather than when a visually attractive backtest appears.

PART III

Evaluation

8 Quote-backed labels

Marks and fills remain separate. `ScenarioOptionPrice` records a declared mark and optional bid and ask. `DeclaredMark` uses the mark as a synthetic or proxy fill; `CrossAtTouch` buys at the ask and sells at the bid, failing closed when the required side is absent. Neither convention establishes that an order executed.

The public repository does not ship historical quote-backed rankings. A future ranking experiment should compare declared marks, quote midpoints, and executable-side conventions, record displayed-size and fill limitations, and report sensitivity to the spread rather than importing unpublished correlations into this paper.

9 Splits and metrics

No train/validation/test market cohort is included in Volt. A future study must use chronological splits and, where repeated issuers are present, an issuer-aware split so company behavior cannot cross freely between training and evaluation.

Primary measurements include:

- precision and normalized discounted cumulative gain at k;
- recovery of exhaustive-search top-decile candidates;
- rank correlation and top-k overlap across execution assumptions;
- distribution calibration and tail-risk error;
- diversity and canonical uniqueness of acquired candidates;
- out-of-time utility after declared costs;
- simulator calls and elapsed compute required to reach a target score.

Invalid-action rate should remain zero after masking. Volt now validates the alignment and integrity of the compiler's structural mask; a production study must compose it with caller-owned liquidity, margin, and account policy.

10 Research sequence

10.1 Canonical search

The bounded enumeration supplies ground truth for raw constructions, exact portfolios, payoff classes, dominance, and clustering. It supports direct measurement of reduction ratios and lets the ranker be compared with exhaustive search before acquisition becomes necessary.

10.2 Event portfolios

An earnings study expands the cohort to liquid U.S. names observed immediately before scheduled releases. Delta buckets and expirations remain narrow enough to preserve an inspectable universe. Post-event labels use executable quote conventions, and time and issuer splits limit memorization.

10.3 Volatility dynamics

Further studies separate fixed-contract changes from relative-coordinate changes in strike and maturity. Term structure, forward variance, skew carry, and spot–volatility correlation are estimated by regime and observation horizon. Their features must be constructed entirely from information available at the ranking clock.

10.4 Conformance

The current Rust tests verify exact canonicalization, terminal hinge payoff, exercise and assignment consequences, deterministic replay, quote authority, scenario fills, graph export, and candidate branching. Future macro support for *verticals*, *straddles*, *strangles*, *butterflies*, *iron condors*, and *calendars* will require independent expansion and ordering fixtures. Such controls distinguish a model failure from a financial representation error.

11 Evidence and limits

Every prediction links to the graph and checkpoint that produced it. Graph attributions and feature ablations describe the model; leg and scenario attribution reconcile the financial outcome; the Volt journal establishes the contracts, observations, and transitions. Keeping those records separate prevents a persuasive model explanation from substituting for accounting evidence.

The grammar, market cohort, simulation model, and objective place hard limits on discovery. A learner may exploit an optimistic fill convention, a data revision, a simulator defect, or a narrow market regime as readily as it exploits a durable relationship. Candidates selected for further study therefore face adversarial scenarios, alternative costs, capacity analysis, and untouched time periods before their apparent utility receives an economic interpretation.

The current synthetic fixtures verify infrastructure only. Their objective and scenario weights are declared synthetic inputs, and their cross-at-touch behavior is a modeled fill rule. The next useful research result is an out-of-time acquisition curve on a public, point-in-time cohort, compared with the full baseline suite at equal evaluation budgets.

12 Conclusion

Volt supplies a stable financial search space; the discovery system learns how to traverse it economically. This study keeps the decision static so candidate identity, labels, rankings, and acquisition cost can be audited against exhaustive ground truth. Once that loop is reliable, the same graph encoder may support sequential control, but trajectory generation, delayed reward, and rollout allocation belong to a different experimental problem.

References

[1] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. Smola, *Deep Sets*, Advances in Neural Information Processing Systems, 2017, https://arxiv.org/abs/1703.06114.

[2] P. Battaglia et al., *Relational Inductive Biases, Deep Learning, and Graph Networks*, 2018, https://arxiv.org/abs/1806.01261.

[3] M. Fey and J. E. Lenssen, *Fast Graph Representation Learning with PyTorch Geometric*, ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019, https://arxiv.org/abs/1903.02428.

[4] J. You, R. Ying, and J. Leskovec, *Design Space for Graph Neural Networks*, Advances in Neural Information Processing Systems, 2020, https://arxiv.org/abs/2011.08843.